

Pythonプログラミング 基礎&実践

総復習演習編

前回の課題振り返り：全体総評

前回の提出課題、皆さん素晴らしい取り組みでした！自力で考えて動かそうとする姿勢がからとてもよく伝わります。

Good： アルゴリズム（FizzBuzzの分岐、最大値、平均、ソート）のコアロジックを人が正しく理解できています。

本日のアプローチ： 各問題の標準的な「解答例」を確認しながら、皆さんのコードで
た「さらによくなる改善ポイント」を順番に教理していきたいと思います

【問題1：FizzBuzz】 解答例

3の倍数、5の倍数、15の倍数を正しく判定する基本的な構成です。

```
for i in range(1, 16):  
    if i % 15 == 0:  
        print("FizzBuzz")  
    elif i % 3 == 0:  
        print("Fizz")  
    elif i % 5 == 0:  
        print("Buzz")  
    else:  
        print(i)
```

【問題1】 振り返りと改善ポイント

条件分岐の順番： `if i % 3 == 0`などを最初に書くと、15の時にそちらに吸い込まれて
"Fizz" と判定されてしまいます。最も厳しい条件（15の倍数）を**最初**に判定しましょ

不要なカウントアップ： `for i in range(...)` のループ内で、`i = i + 1` と手動
算しているコードがありました。for文は自動で次の値に進むため、手動の加算は不要
グの原因）です。

出力の形式： `print(i, "FizzBuzz")`のように、数字と文字列を同時に表示して
がありました。問題の指定（"Fizz"などの文字列のみを表示）に合わせる習慣をつけま
う。

【問題2：最大値探索】 解答例

「暫定チャンピオン」を決めて、ループで順次比較するアルゴリズムです。

```
numbers = [12, 45, 7, 23, 56, 19]

# 最初の要素を暫定最大値にする
max_value = numbers[0]

for n in numbers:
    if n > max_value:
        max_value = n # 記録更新

print(max_value)
```

【問題2】 振り返りと改善ポイント

初期値の選び方（最重要）： `max = 0` からスタートしているコードがいくつかありま
もしリスト内が「すべてマイナスの値（例: `[-10, -5, -20]`）」だった場合、最大値
と誤判定されてしまいます。初期値は `0` ではなく、**リストの最初の値（`numbers[0]`**
ましょう。

汎用的なループ設計： `range(0, 6)` のように要素数を「6」と固定して比較するの
く、`for n in numbers:` と書くことで、リストの長さが変わってもそのまま動く
性の高いコード」になります。

英語の変数名： `for name in numbers:` のように、数値が入っているループ変数を使うと、読む人が混乱します。`num` や `n` など適切な名前を使いましょう。

【問題3：平均値計算】 解答例

要素の合計値と、個数（リストの長さ）を求めて平均値を計算します。

```
numbers = [12, 45, 7, 23, 56, 19]

total_sum = 0
count = 0

for n in numbers:
    total_sum += n
    count += 1

average = total_sum / count
print(average)
```

【問題3】 振り返りと改善ポイント

予約語（組み込み関数）の上書きに注意： 変数名に `sum = 0` を使用しているコードでした。Pythonには最初から `sum()` という合計を計算する便利な関数があります。に `sum` を使うとこの関数が壊れてしまい、以降使えなくなります。変数名は `total_sum` や `total` にしましょう。

計算の位置： ループの内部で毎回 `Average = total / count` を上書き計算していましたが。結果は同じになりますが、ループが終わった後に**外側で1回だけ割り算**の方がスマートで効率的です。

組み込み関数の活用： `total = sum(numbers)` や `len(numbers)` を使って1行で
り解いた素晴らしい実装もありました。基本ループと標準関数の両方をマスターして
しょう。

【発展課題：データの選別】 解答例

```
import random

# 1. 乱数を100個生成
data = [random.randint(1, 100) for _ in range(100)]

# 2. 自作バブルソート（昇順に並び替え）
n = len(data)
for i in range(n):
    for j in range(0, n - 1 - i):
        if data[j] > data[j + 1]:
            data[j], data[j + 1] = data[j + 1], data[j] # スワップ

# 3. 上位50個をスライスで抽出 & 4. ランダムに2個抽出
top_50 = data[50:]
result = random.sample(top_50, 2)
print(result)
```

【発展】 振り返り：バブルソートとスワップ

バブルソートの比較対象： バブルソートは、必ず「隣り合う2つ（`data[j]` と
`data[j+1]`）」を比較して入れ替えます。離れた位置（`data[n]` と `data[m]`）を比
換しているコードがありましたが、これはバブルソートではなく選択ソートに近い別
ゴリズムになってしまいます。

安全な値の交換（スワップ）： データを入れ替える時、tmp を使って片方の値を一時的に保存する必要があります。退避の手順が違うと、値が上書きされて消える原因になります。

Pythonの便利機能： Pythonでは `a, b = b, a` と書くだけで tmp を使わずに一瞬置き換えができるため、この書き方に慣れておくと非常に便利です。

【発展】 振り返り：スライスと標準メソッド

スライスを活用しよう： ソート済みデータから後半50個を抽出する際、手動の for で新しいリストにコピーしているコードがありました。Pythonでは `data[50:]` とスライスを書くだけで簡単に、高速に抽出ができます。

標準的な関数の呼び出し方： リストの長さを取得する際、`data.__len__()` とシス部のメソッドを直接呼んでいるコードがありました。これは間違いではありませんが Pythonでは組み込み関数の `len(data)` を使うのが標準ルールです。

要素の追加： リストに新しい要素を入れるときは、`insert(i, val)` よりもシンプルに追加できる `append(val)` の方がエラーが起きにくく推奨されます。

【演習1】 条件付きリスト生成

1から50までの整数の中から、「4の倍数」かつ「6の倍数ではない」数だけを抽出し、面に表示するプログラムを作成してください。

【演習2】 コイン投げシミュレーション

課題内容

「表 (+10点)」か「裏 (-5点)」がランダムに出るコイン投げを20回繰り返してください。

初期の持ち点を100点としたとき、最終的な持ち点を計算して表示するプログラムを作成してください。

【演習3】 段階的割引の団体料金計算関数

課題内容

団体客の合計入場料を計算して返す関数 `calc_group_fee()` を作成してください。ルールは以下の通りとします。

最初の20人までは通常料金（1,000円/人）

21人目から50人目までの人数分については「2割引（800円/人）」を適用

51人目以降の人数分については「3割引（700円/人）」を適用

※ 関数を作成後、人数が 15、35、60 の場合でそれぞれ呼び出し、合計金額を表示してください。

【演習4】 JSONからの特定データ抽出

課題内容

外部ファイル `employee_data.json` を読み込んでください。

部署（`department`）が "Sales" の社員の「名前（`name`）」と「給与（`salary`）」データをループ処理で画面に表示してください。

【演習5】 JSON形式データの特定条件集計

課題内容

外部ファイル `product_data.json` を読み込んでください。

「在庫数 (stock) が10個未満」の商品の「商品名 (name)」をすべて表示してください。

最後に、それらの該当する商品だけの「在庫数の合計」を計算して表示してください。

【演習6】 デバッグ（プログラムのバグ修正）

ランダムに生成した10個の点数リスト `scores` から、「60点以上のデータだけ」を対象に平均点を計算するプログラムですが、意図した通りに動きません。

バグのあるプログラム：

```
import random
scores = [random.randint(0, 100) for i in range(10)]
print("生成された点数:", scores)

total = 0
count = 0

for s in scores:
    if s >= 60:
        total += s
        count += 1

average = total / count
print("60点以上の平均点:", average)
```

【演習7】 総合応用（待ち時間シミュレーション）

プログラミング部が文化祭に出展するゲームの待ち時間を計算します。

昨年の来訪者の到着時刻： `arrival_times = [0, 3, 4, 10, 11, 12]`

1人あたりの体験時間： **3分**

全員の中での「**最長待ち時間**」を求めて表示してください。

※ 1人目の開始は 0、終了は 3 です。

※ 開始時刻は「**自身の到着時刻**」と「**直前の人**の終了時刻」のどちらか遅い方（大きい方）となります。

※ 各自の待ち時間は「**自身の開始時刻** - **到着時刻**」です。

※ 各自の終了時刻は「**自身の開始時刻** + **体験時間**」です。

【演習8】 総合応用（タスク割り当て）

工芸品の製作タスクを部員3人で分担して製作します。

各工芸品の製作日数： `work_days = [4, 1, 3, 1, 3, 4, 2, 4, 3]`

部員3人の次の作業開始日リスト： `next_start_days = [1, 1, 1]`（初期値は1）

各工芸品は、その時点で「**最も早く手が空く部員**」が順番に担当します。

担当が決まったら、その部員の次の開始日を「**現在の開始日 + 製作日数**」に更新します。

課題：すべての製作が終了する日（全員の作業が終わる最も遅い日）を表示してください。